Township: Technical Design Document

Gated township resident app for maintenance, complaints and visitors (client brief 5). Core build slice.

Author: Aafiya Nawab

GitHub: https://github.com/Aafiyanawab/township-app

Live app: https://township-app-rng1.onrender.com/#/login

The live app runs on a free plan: the first load after inactivity can take about a minute. Demo logins are in deployment.md (for example resident 9000020002, OTP 123456).

1. What I built

I built the core slice the brief asks for, end to end: resident login, maintenance bills with test payment, bank statement matching, complaints with assignment and status, and a committee view. It has 7 screens and four roles (resident, help desk, accounts, committee). The hard part named in the brief, payment matching from a bank CSV, is the most carefully built piece. The AI part is complaint category and urgency.

I chose depth on the core slice over breadth: visitor management, amenity booking and Marathi/Hindi are not built (see section 9).

2. Architecture and why

Layer	Choice	Why
Frontend	React + TypeScript + Vite + Tailwind, hash routing	Fast to build, typed, mobile-first. Hash routes (/#/login) never clash with API paths, so one server can serve both.
API	FastAPI + Pydantic	Input validation by default, clear route/role structure, and I know it well. Business rules live in plain service modules, so they are unit-testable without HTTP.
Database	SQLAlchemy 2. SQLite on the free Render plan, PostgreSQL supported (docker-compose for local)	Money is Numeric/Decimal. SQLite keeps the free demo self-contained. Moving to Postgres is a DATABASE_URL change.
Auth	Mock OTP login, JWT (PyJWT) with exp, role read from the database on every request	The brief says mock outside systems. Server-side role lookup means a deactivated user or changed role takes effect immediately.
AI	Gemini (optional) behind a rule layer, keyword fallback	The safety rule must never depend on a model. The app works fully with no API key.
Delivery	Docker (multi-stage), Render, GitHub Actions	One service: FastAPI serves the built frontend, so there is no CORS setup in production. CI runs backend tests and the frontend lint and build on every push.

Request path: browser (React) to FastAPI (auth, role check, validation) to service modules (billing, matching, complaints) to SQLAlchemy to the database. Complaint text may also go to the LLM, and its answer is validated before use.

3. Data model

Table	Purpose and key fields
flats	Code (e.g. T12-0408), tower, area_sqft, registered upi_id (used by matching).
users	Phone, name, role (resident, helpdesk, accounts, committee), flat_id, is_tenant, is_active.
bills	One per flat per period. amount = area x rate, paid_amount, issued_on, due_on. Unique (flat, period).
payments	Amount, method, source (online, bank_match, manual), bill_id, bank_row_id.
bank_rows	One CSV line. Unique row_hash (makes re-upload safe), amount, reference, upi_id, status, suggested flat, confidence, match_reason.
ledger_entries	Per-flat ledger: bills are positive, payments negative.
complaints	Text, category, urgency, status, assigned_to, ai_category, ai_confidence, safety_escalated, duplicate_of_id, rating, respond_by, first_response_at.
complaint_events	Timeline of each complaint (created, assigned, status, category_changed, linked_duplicate, reopened) so residents see where it stands.

Rules: money is always Decimal, never float. A resident can only reach rows for their own flat; another flat's bill or complaint returns 404 so its existence is not revealed. Payment, bill update and ledger entry are written in one transaction.

4. Main flows

4.1 Login

Phone number, then a mock OTP (123456 only when OTP_MODE=mock). Unknown phone and wrong OTP return the same message, so phone numbers cannot be enumerated. The JWT carries user id and expiry; role and active status are re-read from the database on each request. OTP attempts are rate-limited per phone and per IP.

4.2 Bills and test payment

Bill = area x Rs 6.50 per sq ft (the three sample rows in the brief reproduce exactly: 7,475, 6,370 and 10,660). Late fee is simple interest on the outstanding balance for the days overdue (assumptions A1 to A3). A test payment is rejected if the amount is zero, negative or above what is owed; it is applied to the oldest bill first (A4) and recorded with its ledger entry in one transaction.

4.3 Bank statement matching (the hard part)

Accounts uploads a CSV (date, description, reference, upi_id, amount). Each credit row is scored against every flat on three signals:

Signal	Weight
Reference or description contains the flat code (T12-0408, T120408 and T12 0408 all normalise)	60
UPI ID equals the flat's registered UPI ID (case-insensitive)	30
Amount equals something the flat owes (an open bill, the total principal, or principal plus late fee)	20

Outcome	Rule	Effect
matched	Best score >= 80, no competing flat, amount not above what the flat owes	Posted automatically
queued	Score 40 to 79, or a second different flat scores >= 30, or amount exceeds dues	Accounts queue, never posted automatically
unmatched	Best score < 40 (for example amount only, or a typo in the flat code)	Accounts queue
ignored	Debit or zero amount	Not a payment received

Further safeguards: two payments for the same bill in one file cannot both auto-post; every row has a stable hash, so uploading the same file twice posts nothing new (the sample file reports 8 duplicates skipped on the second upload); a CSV with any invalid line is rejected whole; only accounts staff can resolve a queued row by choosing a flat, and a resolved row cannot be posted twice. The sample statement in the repo produces 2 posted, 3 queued, 2 unmatched and 1 ignored.

4.4 Complaints

A resident writes free text. The system classifies it (section 5), links it to an earlier complaint if it is a duplicate, auto-assigns it to the help-desk user with the fewest active complaints, and sets a response timer by urgency. The help desk can change the category in one call, assign, and move the status (open, assigned, in progress, resolved, closed). A resolved complaint can be reopened by the resident or rated 1 to 5, which closes it. Reopens are recorded as events so the committee view can report the reopened percentage, the brief's success metric.

4.5 Committee view

One endpoint feeds the dashboard: collections against billed, dues ageing (0 to 30, 31 to 60, 61+ days, principal only), complaints by status, average hours to first response, reopened count and percentage, and average rating.

5. How the AI part works

Three layers, in priority order:

- **Safety rule (always first).** Fire, smoke, gas, lift stuck or trapped, and electric shock or sparking force urgency to critical, escalate to the duty manager and set a one-hour timer, whatever the model says. Tests include a fake model that answers low urgency and the rule still wins.
- **LLM (optional).** If a Gemini key is configured, the model returns category, urgency and confidence as JSON. The complaint is sent as untrusted text; the reply is parsed and checked against fixed allowlists, so text such as "ignore your instructions" cannot change the outcome.
- **Keyword fallback.** Used when there is no key, or the call errors, times out or returns anything invalid. The app works fully without any AI.

When the AI is wrong or unsure	What the system does
Confidence below 60	Category becomes General, the complaint is flagged Needs review, and the help desk changes the category in one tap.
Model error, timeout or invalid output	Falls back to keyword rules and records that fallback was used.

When the AI is wrong or unsure	What the system does
Safety words present	Rule overrides the AI: critical, duty manager, one-hour timer.
Many residents report the same lift	Complaints are linked to the original (same tower, 48 hours), never auto-closed.
Help desk disagrees with the category	One-call override, recorded in the timeline, and the complaint is marked as overridden.

Honest note: in my own testing, "call the plumber and fix the pipeline" was classified General and flagged Needs review. That is the unsure path working as designed, and it also shows the keyword vocabulary is small. Adding more terms is on the next-steps list.

6. Security and testing

- Authentication on every route except /health and the two OTP routes; a test iterates all routes and fails if a new route has no auth dependency. API docs are behind login.
- Role checks on the server for resident, help desk, accounts and committee; residents are limited to their own flat (404 for others).
- Validation: Pydantic limits on every body (phone format, complaint 5 to 1000 characters, rating 1 to 5, status and category allowlists); CSV upload limited by type and size, parsed all-or-nothing.
- Rate limiting on OTP routes (per phone and per IP). Security headers (nosniff, frame deny, referrer policy, a same-origin Content-Security-Policy, permissions policy). CORS limited to configured origins.
- Generic 500 responses with no stack traces; no secrets in the repository; production refuses to start with a weak JWT secret. SQL goes through SQLAlchemy only.
- Dependencies: I replaced python-jose with PyJWT and upgraded FastAPI and python-multipart. pip-audit reported no known vulnerabilities and npm audit reported none for production dependencies at the time of writing.
- Tests: over 160 automated tests (pytest) cover billing and late fees, matching and CSV import, complaint rules and lifecycle, auth, access control and the endpoints. GitHub Actions runs them, plus the frontend lint and build, on every push.

7. UI and design decisions

The brief asks for minimal design, a restrained palette and nothing without a purpose, and says residents open the app for seconds. So the resident home leads with one dues card (amount, due date, days overdue, late fee and a single Pay button), then open complaints, with a bottom navigation bar for everything else.

- **Palette:** warm cream background, white cards, forest green as the primary colour, and marigold used sparingly as the only accent. I avoided the blue and purple gradient look. Colours are design tokens in one CSS file.
- **Type:** Bricolage Grotesque for headings and DM Sans for body text, bundled with the app, so no external font requests on slow connections.
- **Mobile first:** large tap targets, single-column layouts, bottom navigation. Every data screen has loading skeletons, an empty state and an error state with a retry button.
- **Honest display:** money is shown from exact decimal strings; overdue days, late fee and total are shown separately so the resident can see how the total is made up.

Screen	Role	Why it exists
Login	All	Phone then OTP, with the demo OTP shown so a reviewer can get in.
Resident home	Resident	Dues, open complaints and one Pay action in two taps or fewer.
Bills and pay	Resident	Bill list, ledger, test payment with confirmation and receipt.
Complaints	Resident	Raise one, follow the timeline, rate or reopen.
Help desk queue	Help desk	Sorted by urgency with overdue and duplicate badges; one-tap category change, assign, status.
Accounts	Accounts	CSV upload with a result summary, the review queue with match reason and confidence, resolve or ignore.
Committee dashboard	Committee	Collections, dues ageing, complaint times, reopened percentage and rating, shown as simple bars.

Not yet done: a final visual polish pass (an illustration style for empty states, a navigation bar that does not overlap content on wide screens, and hiding the overdue label on a paid bill).

8. Assumptions (labelled)

The brief does not give these, so I made them explicit. Each is easy to change in code.

ID	Assumption
A1	Late fee is 18% per year simple interest. The brief says per bye-laws but gives no rate.
A2	Interest starts the day after the due date (no grace period).
A3	Interest is calculated on today's outstanding balance, not on each past day's balance.
A4	A payment is applied to the oldest bill first; any extra becomes an advance credit.
A5	Bills are issued on the 1st and due on the 10th, based on "most bills are paid in the first 10 days".
A6	Days overdue are computed from today's date, so the sample rows show more days than the brief's 47 and 18.
A7	SMS OTP and payments are mocked. In mock mode the OTP is always 123456 (demo only).
A8	All names, phone numbers and UPI IDs in the seed data are fictional.
A9	A payment that clears the principal also settles the accrued late fee; late fee is not stored separately.
A10	Matching weights are reference 60, UPI ID 30, amount 20. Auto-post needs a score of 80 or more.
A11	Scores of 40 to 79, or conflicting signals, go to the accounts queue. Below 40 shows as unmatched.
A12	Bank CSV columns are date, description, reference, upi_id, amount (credit positive, debit negative).
A13	Response timers: critical 1 hour, high 4 hours, normal 24 hours, low 48 hours.
A14	The duty manager is the lowest-id help-desk user; other complaints go to the least-loaded help-desk user.
A15	Safety words always escalate; a false alarm is accepted over a missed emergency.

ID	Assumption
A16	Duplicates are matched by category and tower (shared areas) or by flat (in-flat issues) within 48 hours.
A17	The free Render plan uses SQLite that is reset on restart, so demo data is re-seeded at start (SEED_ON_START).

9. What I cut, known limits and next steps

Cut on purpose

- Gate visitor management (guard tablet, photos, guest codes, 90-day photo deletion), amenity booking, Marathi and Hindi, family and tenant member management, notices. All are optional depth in the brief; the core slice comes first.
- Complaint photo upload (the model has a field, there is no upload yet). Real SMS and payment gateway (mocked, as the brief allows).

Known limits

- Free plan: the service sleeps after inactivity (first load can take about a minute) and the SQLite data resets on restart; the seed data is restored automatically.
- The OTP rate limiter is in memory, so counters are per process and reset on restart. OTP is a mock and the demo OTP is public by design.
- Late fee uses simple interest on today's balance (A3). The keyword fallback has a small vocabulary. The Gemini free tier may use inputs to improve Google products, so only fictional data should be sent.

Next steps

- Move to PostgreSQL in production with Alembic migrations; Redis-backed rate limiting; real SMS OTP.
- Visitor management with offline-tolerant guard tablet screens and automatic photo deletion; amenity booking; Marathi and Hindi.
- Visual polish pass; offline-capable installable PWA; a monthly committee report export.
- Evaluate the complaint classifier on a labelled set of real complaints and tune the confidence threshold.

10. How I used AI, and what went wrong

I used AI tools throughout. I used Claude to plan the scope, design the matching and complaint rules and write the core service modules with their tests, and Codex to generate endpoints and screens from prompts with a project context file (AGENTS.md). After each generated step I ran the tests, read the review output and committed in small steps. The scope cuts, the thresholds, the assumptions and the final choices are mine, and I can explain each.

- CI failed on the first auth commit because the tests depended on a database I had seeded by hand. I fixed CI first, then made the tests build their own data.
- The Vite template installed a TypeScript version that conflicted with the ESLint plugin; I pinned TypeScript to a compatible version.
- A dependency audit found 38 advisories. I replaced and upgraded the affected packages one at a time, with the tests as a safety net.